



Engenharia de Software Conceito – parte 2

Profa. Patricia Rucker de Bassi

Ciclo de vida de desenvolvimento

- Engloba todos os processos, tarefas e atividades desde a concepção do software até sua descontinuidade ou substituição por outro.
- Funciona como um esquema, mapeando as fases, atividades, entregas e responsabilidades de cada recurso envolvido no processo de desenvolvimento do software.

Ciclo de vida de desenvolvimento

- Benefícios
 - Visualizar a real necessidade do software
 - Planejá-lo melhor
 - Disponibilidade para melhorar o desempenho
 - Realizar correções antecipadas
 - Facilita a futura manutenção
 - Localizar erros com mais facilidade
 - Diminui custo
 - Permite garantir a qualidade do produto final

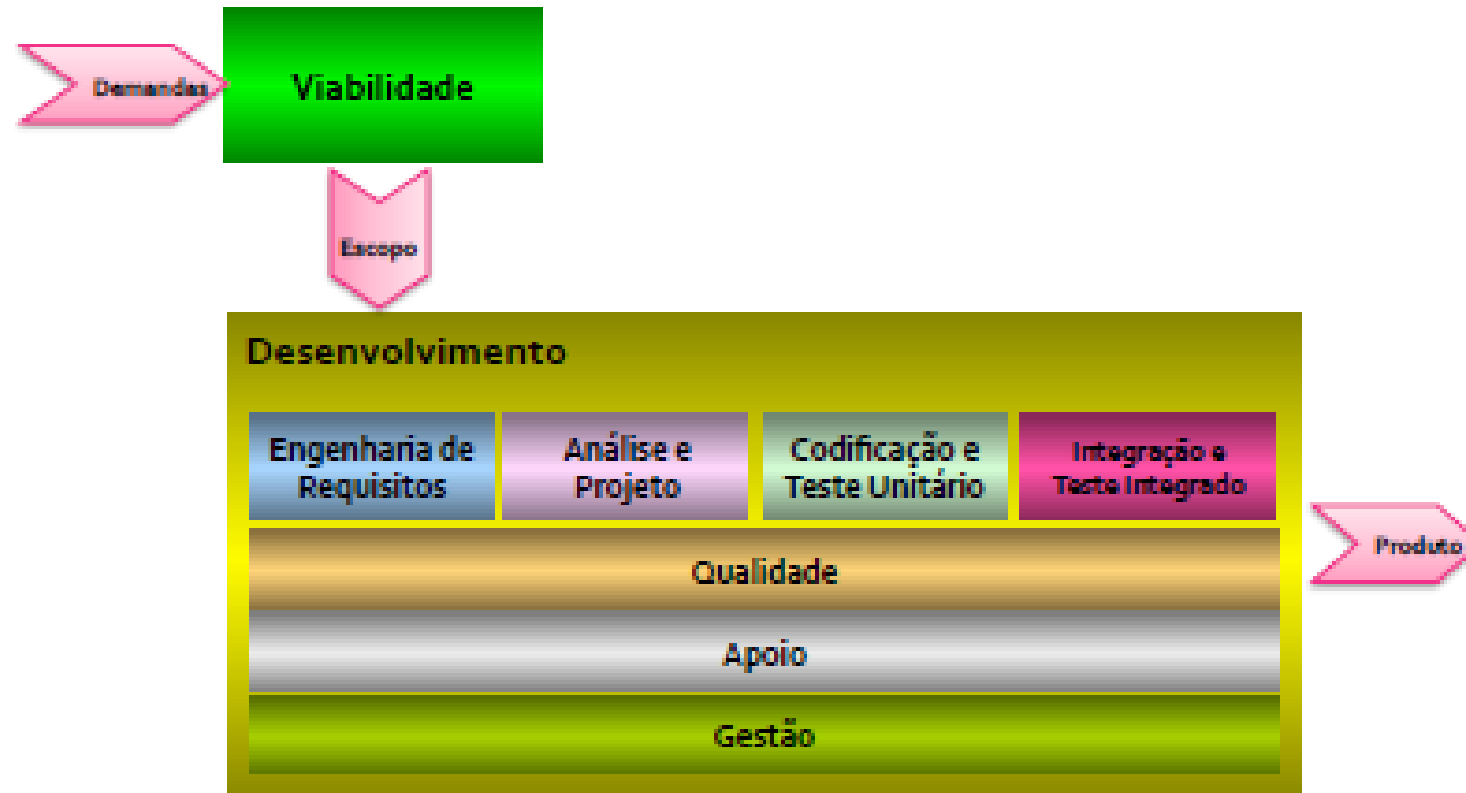
Ciclo de vida do desenvolvimento

- Etapas básicas
 - Definição
 - Conhecer a situação atual, identificar o problema e buscar uma solução
 - Modelagem dos processos e a análise do sistema
 - Escolha do **modelo de ciclo de vida** do desenvolvimento
 - Desenvolvimento
 - Projeto, prototipagem, codificação, testes e integração (se houver)
 - Deve considerar as regras de negócio propostas na etapa anterior
 - Operação
 - Treinamento aos usuários
 - Correção de erros
 - Continuidade do software (novos requisitos e/ou funcionalidades)
 - Depende do **modelo de ciclo de vida** do desenvolvimento que foi escolhido

Ciclo de vida de produto de software



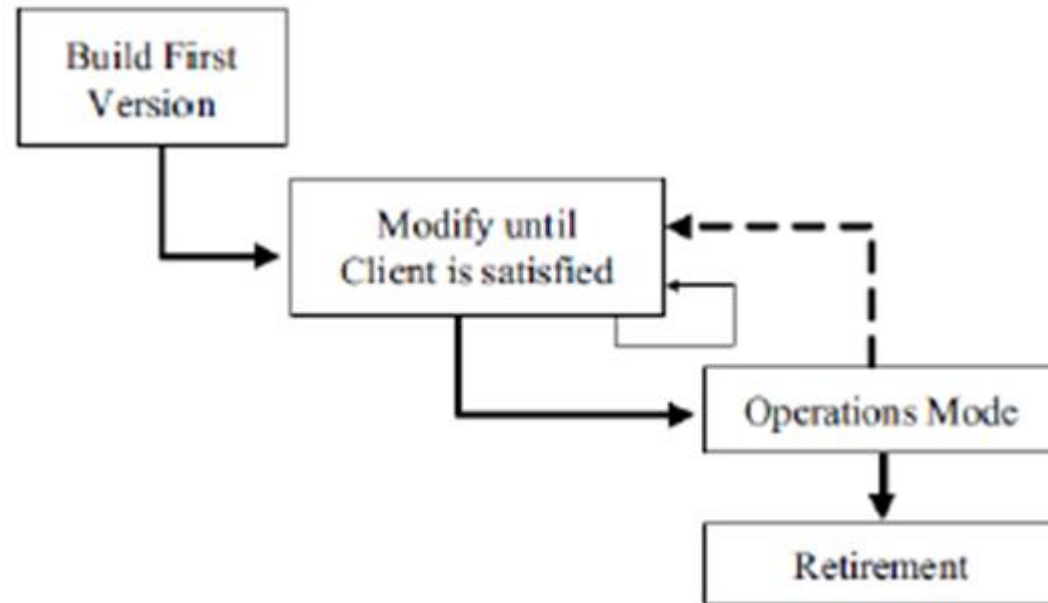
Ciclo de vida de desenvolvimento



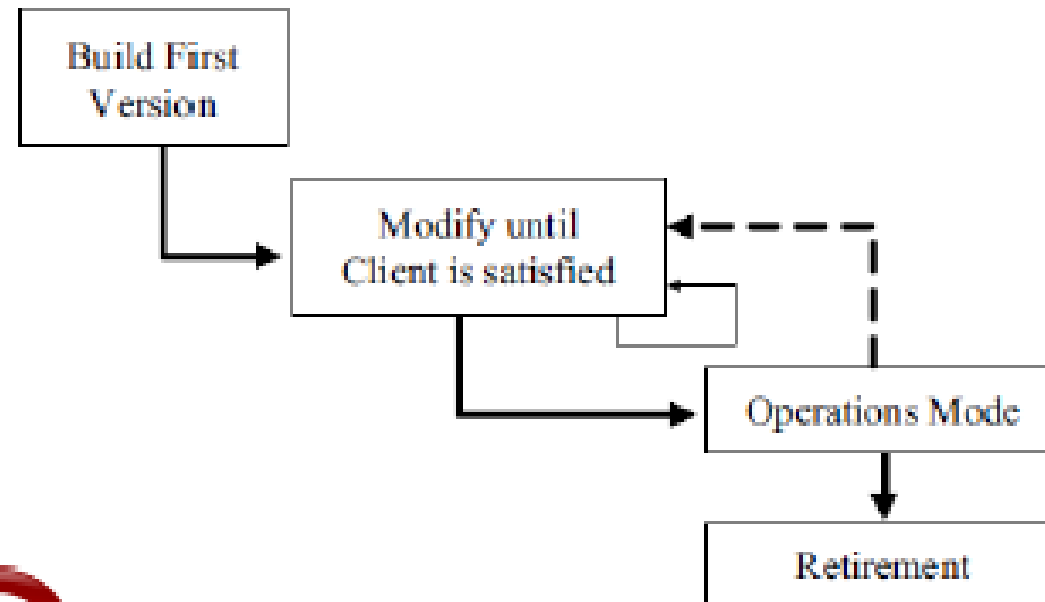
Modelos de ciclo de vida

- Podem ser entendidos como:
 - A forma como as etapas de desenvolvimento são organizadas;
 - A sequência em que as atividades dos projetos de desenvolvimento e manutenção de software são executados.

Codificar - Testar



Codificar - Testar



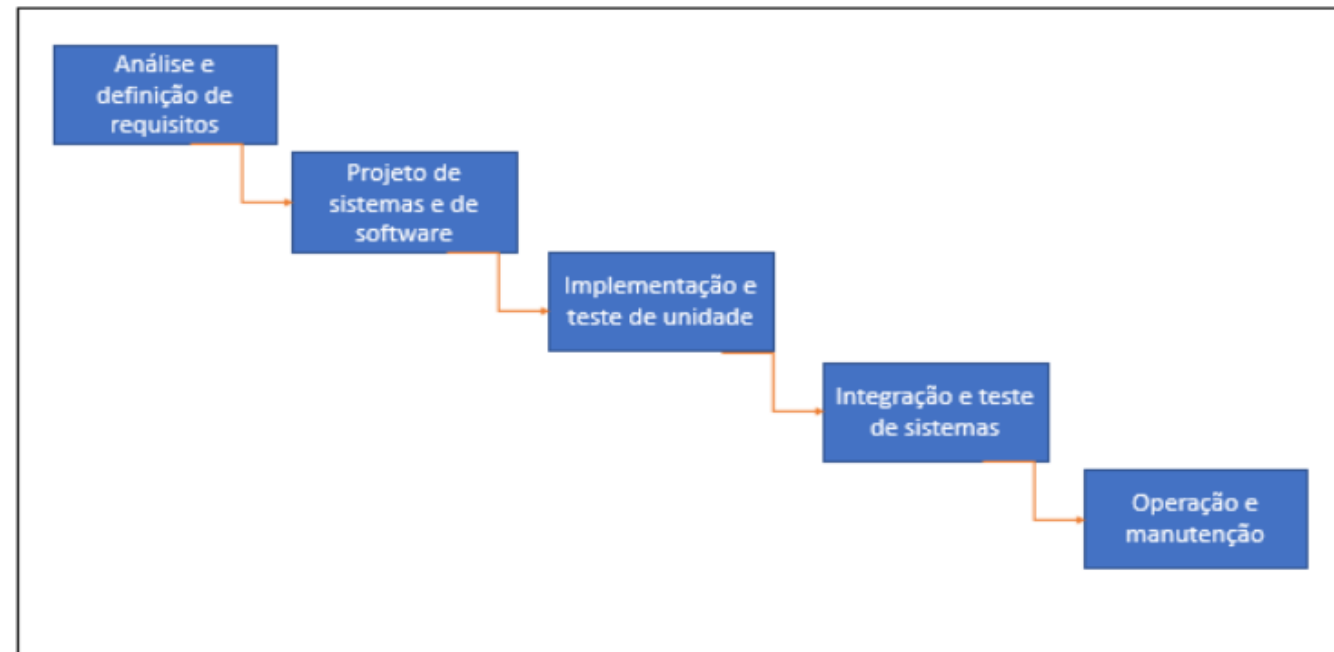
Isto não é um ciclo de vida!!!

Modelos de ciclo de desenvolvimento de software

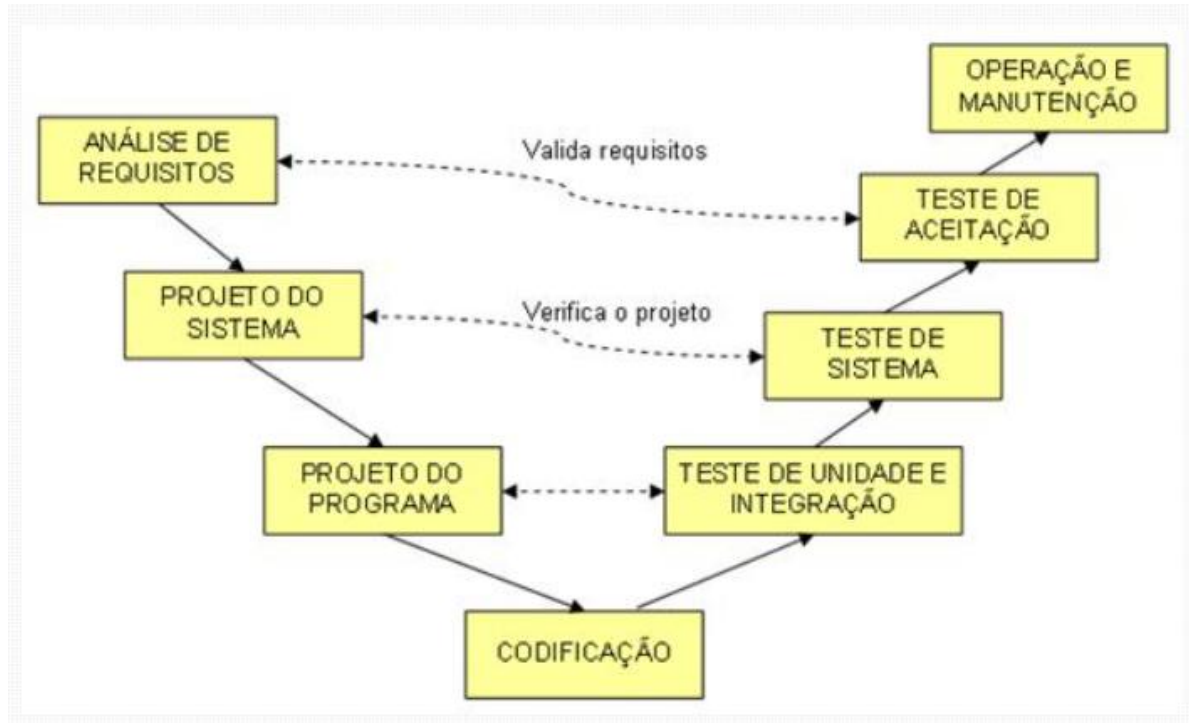
- Modelagem Tradicional
 - Modelo Cascata ou Clássico
 - Modelo V
 - Modelo Incremental ou Iterativo
 - Modelo Evolutivo
 - Modelo Espiral (Barry Boehm)
 - Modelo Prototipação

Modelo Cascata ou Clássico

- O modelo cascata é um modelo tradicional que foi criado em 1966, porém, formalizado somente por volta de 1970.
- Este modelo define que as fases serão sequenciais, uma fase tem que estar completa antes de passar para a próxima.
- Seu principal benefício é a facilidade de gestão do projeto.
- Já uma desvantagem desse modelo é a dificuldade de acomodar mudanças depois que o processo já está em andamento.



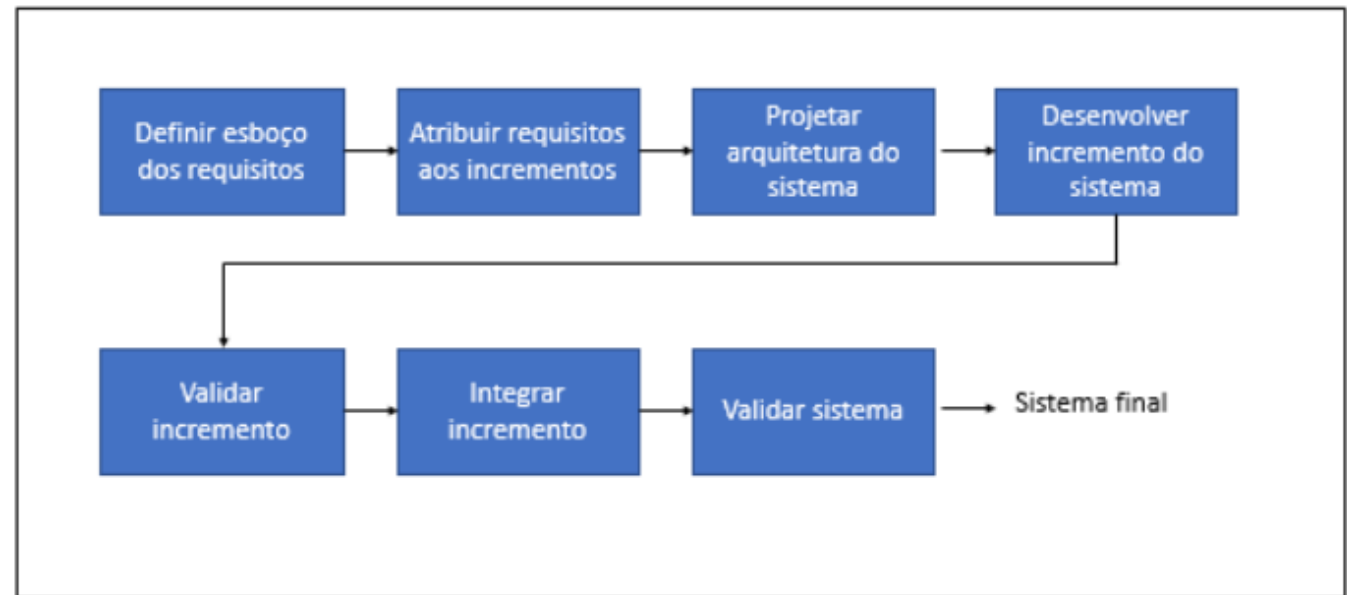
Modelo V



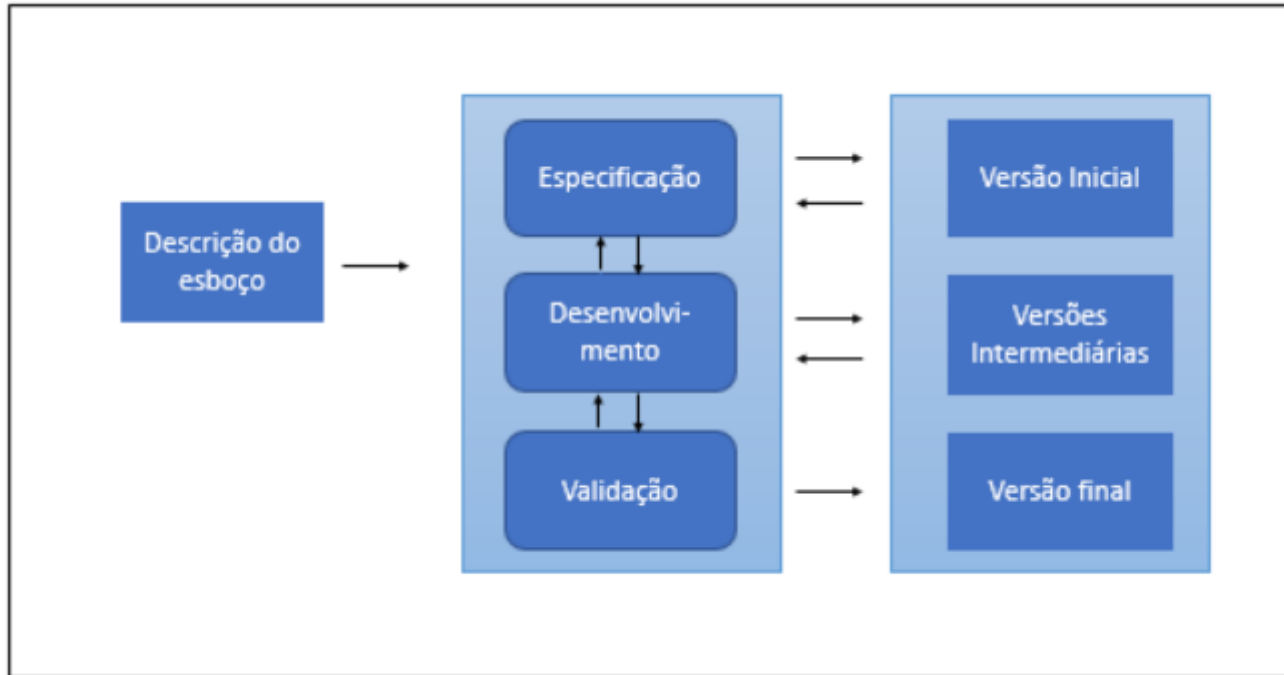
- Neste modelo, do Ministério de Defesa da Alemanha, 1992, o modelo em cascata é colocado em forma de "V".
- A característica principal desse modelo, que o diferencia do modelo em cascata, é a ênfase dada à verificação e validação: cada fase do lado esquerdo gera um plano de teste a ser executado no lado direito.
- Mais tarde, o código fonte será testado, do mais baixo nível ao nível sistêmico para confirmar os resultados, seguindo os respectivos planos de teste: o teste de unidade valida o projeto do programa, o teste de sistema valida o projeto de sistema e o teste de aceitação do cliente valida a análise de requisitos.
- Da mesma forma que o modelo em cascata, o cliente só recebe a primeira versão do software **no final do ciclo**, mas apresenta menos risco, devido ao planejamento prévio dos testes nas fases de análise e projeto.

Modelo Incremental ou Iterativo

- Esse modelo foi criado como uma melhoria do modelo cascata e também é um modelo tradicional.
- Nesse modelo, o desenvolvimento é dividido em etapas, que produzirão o sistema até chegar a sua versão final.
- Este é um modelo ideal caso os requisitos ainda não estejam tão claros. Por exemplo, se algum erro é cometido, apenas o último incremento será descartado.
- Além disso, como o foco é a entrega de cada incremento, a funcionalidade do sistema estará **disponível mais cedo** para o usuário.

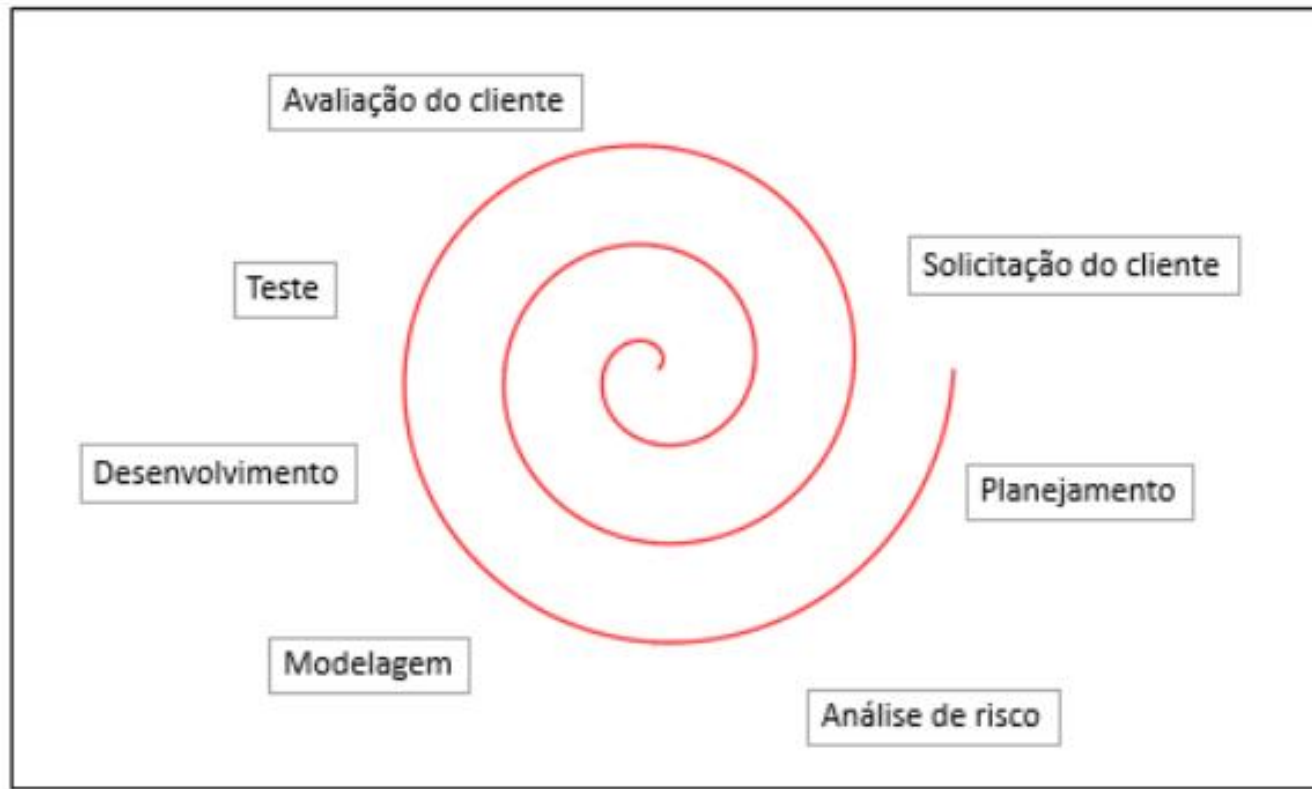


Modelo Evolutivo



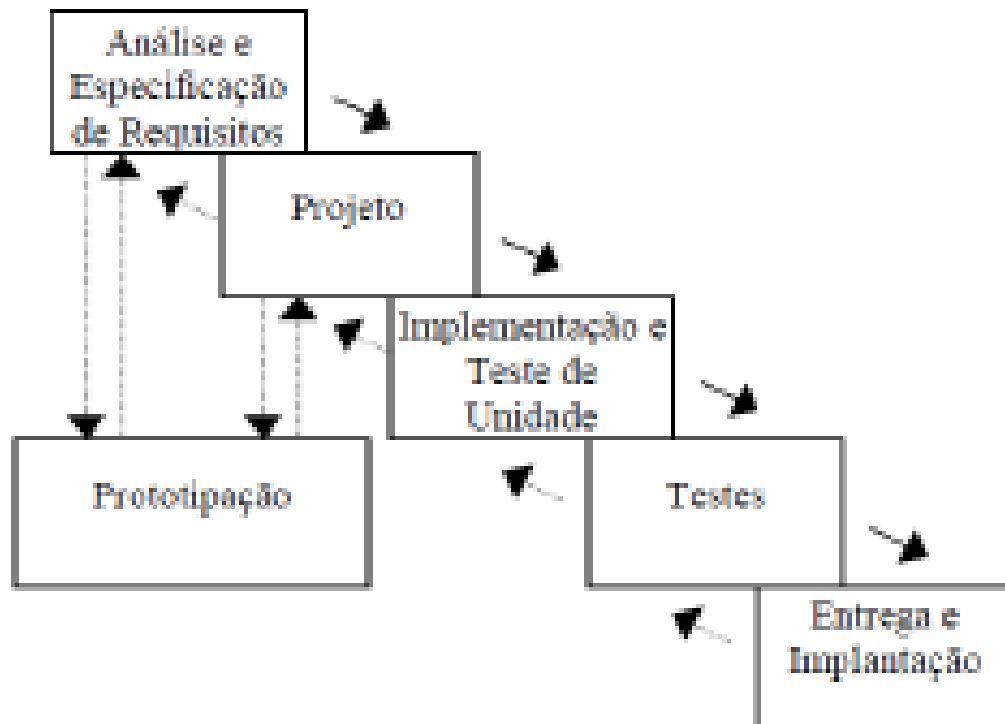
- O modelo evolutivo tem a facilidade de mudanças e a possibilidade de oferecer novas funcionalidades naquele mesmo momento.
- É bastante indicado para sistemas de curto prazo ou sistemas pequenos e médios.
- É um modelo que possui grande interação com o usuário.
- Esse modelo tem a desvantagem de se ter dificuldade de estabelecer limites quanto ao escopo e tempo, o que demanda maior gerenciamento de projeto.

Modelo Espiral (Barry Boehm)



- Nesse modelo o processo não é baseado em uma sequência de atividades com retorno.
- Ele se baseia em uma espiral, onde cada volta nessa espiral representa uma fase no processo.
- Além disso, não existem fases fixas, essas voltas são escolhidas de acordo com o que é requerido.
- Uma desvantagem desse modelo é que ele é melhor aplicado somente em produtos internos da empresa.

Modelo Prototipação



- No ciclo de vida de prototipagem, não é exigido um conhecimento aprofundado dos requisitos num primeiro momento.
- Isso é bastante útil quando os requisitos não são totalmente conhecidos, são muitos complexos ou confusos.
- Assim, o cliente experimentará, na prática, como o sistema ou parte dele funcionará.
- A partir desse primeiro contato, o cliente esclarece o que não foi bem interpretado, aprofunda alguns conceitos e até descobre um pouco mais sobre o que realmente precisa.
- A partir deste feedback, novos requisitos são colhidos e o projeto ganha maior profundidade. Outro protótipo é gerado e apresentado ao cliente, que retorna com mais feedbacks. Ou seja, o cliente participa ativamente do início ao fim do processo.

Critério	Cascata / Clássico	Modelo V	Incremental / Iterativo	Evolutivo	Espiral (Boehm)	Prototipação
Origem / Ano	Criado em 1966, formalizado ~1970	Ministério de Defesa da Alemanha, 1992	Criado como melhoria do modelo Cascata	Década de 80, publicado pela HP em 1996	Proposto por Barry Boehm	Década de 70, vários autores
Característica principal	Fases sequenciais; uma fase deve terminar antes da próxima começar	Cascata em forma de "V"; cada fase da esquerda gera um plano de teste para a direita correspondente	Desenvolvimento dividido em incrementos; cada etapa produz uma versão parcial até a final	Facilidade de mudanças e novas funcionalidades em tempo real; forte interação com o usuário	Baseado em espiral; cada volta representa uma fase; sem fases fixas, definidas conforme necessidade	Não exige conhecimento aprofundado dos requisitos inicialmente; protótipos são criados e refinados com feedback do cliente
Entrega ao cliente	Cliente recebe a primeira versão apenas no final do ciclo	Cliente recebe a primeira versão no final do ciclo (menos risco que o Cascata)	Funcionalidades disponíveis mais cedo, a cada incremento entregue	Entregas contínuas com novas funcionalidades	Entregas dependem das voltas da espiral definidas no processo	Protótipos entregues desde o início para validação com o cliente
Ênfase em testes<	Testes ao final do desenvolvimento	Verificação e validação desde o início; teste de unidade, de sistema e de aceitação planejados nas fases de análise e projeto	Testes a cada incremento; erros afetam apenas o último incremento	Testes contínuos conforme novas funcionalidades	Avaliação de riscos em cada volta da espiral	Validação prática com o cliente a cada protótipo
Principal vantagem	Facilidade de gestão do projeto	Menos risco que o Cascata devido ao planejamento prévio dos testes	Ideal quando requisitos não estão claros; funcionalidade disponível mais cedo; apenas o último incremento é descartado em caso de erro	Flexibilidade para mudanças e novas funcionalidades; alta interação com o usuário	Flexibilidade: fases escolhidas conforme o requerido; avaliação contínua	Cliente participa ativamente do início ao fim; esclarece requisitos confusos ou complexos
Principal desvantagem	Dificuldade de acomodar mudanças após o processo em andamento	Cliente só recebe o software ao final (igual ao Cascata)	Necessita gestão de múltiplos incrementos	Dificuldade de estabelecer limites de escopo e tempo; demanda maior gerenciamento de projeto	Melhor aplicado apenas em produtos internos da empresa	Pode gerar expectativa de que o protótipo é o produto final
Indicação de uso	Projetos com requisitos bem definidos e estáveis	Projetos que exigem rigor na verificação e validação desde o início	Requisitos não tão claros; necessidade de entregas parciais	Sistemas de curto prazo ou pequenos e médios; forte necessidade de interação com o usuário	Produtos internos da empresa; projetos que exigem avaliação contínua de riscos	Requisitos não totalmente conhecidos, complexos ou confusos
Participação do cliente	Baixa — apenas no início (requisitos) e no final (aceitação)	Baixa — no início e no final, mas com testes planejados previamente	Média — feedback entre incrementos	Alta — interação constante com o usuário	Média — nas avaliações de cada volta da espiral	Muito alta — participação ativa do início ao fim
Flexibilidade a mudanças	Baixa — difícil acomodar mudanças em andamento	Baixa — similar ao Cascata, mas com menor risco	Média — mudanças absorvidas no próximo incremento	Alta — mudanças e novas funcionalidades em tempo real	Alta — fases definidas conforme necessidade	Muito alta — requisitos refinados a cada protótipo

Modelo X Método

Modelo de ciclo de vida descreve a estrutura e a sequência das fases do desenvolvimento

- Como as atividades são organizadas no tempo (sequencial, iterativa, espiral, etc.).
- É um esquema abstrato que mapeia fases, entregas e responsabilidades.

Método de desenvolvimento (ou metodologia) é algo mais prescritivo e operacional

- Define como executar cada atividade, com técnicas, ferramentas, papéis e regras específicas.
- Exemplos: Scrum, XP, Cleanroom.

Métodos de desenvolvimento de software

Métodos Tradicionais

- Metodologia RUP (Rational Unified Process)
 - Método estruturado em quatro fases (Concepção, Elaboração, Construção, Transição), com disciplinas e artefatos bem definidos. É iterativo e incremental, mas prescritivo (define desde os testes até modelagem).
- Cleanroom Software Engineering
 - Método focado em prevenção de defeitos em vez de correção, usando desenvolvimento estatístico formal, inspeções rigorosas e testes baseados em probabilidade.

Métodos de desenvolvimento de software

Métodos Ágeis

- Scrum
 - Framework com papéis definidos (Product Owner, Scrum Master, time de desenvolvimento), cerimônias (Sprint Planning, Daily Scrum, Sprint Review, Retrospective) e artefatos (Product Backlog, Sprint Backlog, Increment). Foco em entregas iterativas de 2-4 semanas (Sprints).
- Extreme Programming (XP)
 - Método com foco em qualidade técnica por meio de práticas como TDD (Test-Driven Development), pair programming, integração contínua, refactoring e propriedade coletiva do código.
- Kanban
 - Método de fluxo contínuo com quadro visual (colunas: To Do, Doing, Done), limite de WIP (Work in Progress) e foco em entregar valor de forma contínua, sem iterações fixas.

Métodos de desenvolvimento de software

Métodos Modernos / Híbridos

- DevOps
 - Não é exatamente um método de desenvolvimento isolado, mas uma cultura/prática que integra desenvolvimento e operações com esteiras de CI/CD (integração e entrega contínuas), automação e monitoramento.
- Lean Software Development
 - Inspirado no Sistema Toyota de Produção, foca em eliminar desperdício, amplificar aprendizado, decidir o mais tarde possível e entregar rápido.
- Feature-Driven Development (FDD)
 - Método com foco em funcionalidades do domínio, com modelagem inicial e depois desenvolvimento por feature em ciclos curtos.

Referências

- <https://www.devmedia.com.br/ciclos-de-vida-do-software/21099>
- <https://www.treinaweb.com.br/blog/ciclo-de-vida-software-por-que-e-importante-saber/>
- <https://www.ibm.com/br-pt/think/topics/sdlc>

Atividade

- Pesquisar 1 exemplo de software real desenvolvido para o Modelo de Ciclo de Vida de Desenvolvimento de Software designado para sua equipe.
- Descrever de forma resumida:
 - o que faz o software,
 - qual o Modelo de Ciclo de Vida de Desenvolvimento de Software aplicado
 - e o motivo de ter sido escolhido o modelo de ciclo de vida aplicado.
- Atividade em grupo.
- Utilizar o arquivo colaborativo na equipe do Teams **até o final da aula de hoje!**

Equipes X Modelo

- Modelo Cascata ou Clássico
- Modelo V
- Modelo Incremental ou Iterativo
- Modelo Evolutivo
- Modelo Espiral (Barry Boehm)
- Prototipação